T-Swap Audit Report

Version 1.0

HadoSama

April 10, 2026

T-Swap Audit Report

HadoSama

April 10, 2026

Prepared by: HadoSama Lead Auditors: - HadoSama

Table of Contents

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: Uniswap Explained

Disclaimer

The HadoSama team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: e643a8d4c2c802490976b538dd009b351b1c8dda
- Date: 10/4/2026
- Auditor: HadoSama

Scope

```
./src/
#-- PoolFactory.sol
#-- TSwapPool.sol
```

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

We found 16 issues in the codebase.

Issues found

Severity	Number of issues found
High	5
Medium	1
Low	2
Info	8
Total	16

Findings

High

[H-1] Incorrect fee calculation in TSwapPool::getInputAmountBasedOnOutput causes protocol to take too many tokens from users, resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Proof of Concept:

Recommended Mitigation:

```
function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
-       return ((inputReserves * outputAmount) * 10_000) / ((
    outputReserves - outputAmount) * 997);
```

```
13 +        return ((inputReserves * outputAmount) * 1_000) / ((
       outputReserves - outputAmount) * 997);
14      }
```

[H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens

Description: The swapExactOutput function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`, where the function specifies a minOutputAmount, the swapExactOutput function should specify a maxInputAmount.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept: 1. The price of 1 WETH right now is 1,000 USDC 2. User inputs a swapExactOutput looking for 1 WETH a. inputToken = USDC b. outputToken = WETH c. outputAmount = 1 WETH d. deadline = whatever 3. The function does not offer a maxInput amount 4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 WETH is now 10,000 USDC. 10x more than the user expected 5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC

Paste the test below in the `TSwapPool.t.sol` file

Code

```
1
2  function testSwapExactOutputNoSlippageProtection() public
       depositLiquidity {
3          //
               ============================================================

4          //                          SETUP
5          //
               ============================================================

6          // Simulate: 1 WETH = 1,000 USDC (pool ratio: 100 WETH :
               100,000 poolTokens)
7          // We'll use a fresh pool with that ratio for clarity
8
9          address someUser = makeAddr("someUser");
10         uint256 userStartingBalance = 100_000e18;
11         poolToken.mint(someUser, userStartingBalance);
12
13         vm.startPrank(someUser);
14         poolToken.approve(address(pool), type(uint256).max);
15
```

```
        //
            ============================================================

        //                  RECORD PRE-SWAP STATE
        //
            ============================================================

        uint256 userPoolTokenBalanceBefore = poolToken.balanceOf(
            someUser);

        // User wants exactly 1 WETH out.
        // At current pool price they expect to pay ~1,000 poolTokens.
        // There is no maxInputAmount param to protect them.
        uint256 expectedMaxInput = 1e18; // what user thinks they'll
            pay

        //
            ============================================================

        //              SIMULATE MARKET MOVE (front-run / mempool shift
            )
        //
            ============================================================

        // Attacker or market movement drains WETH from the pool before
        // the user's tx lands, making WETH much more expensive.
        vm.stopPrank();

        address attacker = makeAddr("attacker");
        poolToken.mint(attacker, 50e18);
        vm.startPrank(attacker);
        poolToken.approve(address(pool), type(uint256).max);

        // Attacker swaps in a huge amount of poolToken to drain most
            WETH
        pool.swapExactInput(
            poolToken,
            50e18, // dump 80,000 poolTokens in
            weth,
            1, // accept any amount of WETH out
            uint64(block.timestamp)
        );
        vm.stopPrank();

        //
            ============================================================

        //                  USER'S TX LANDS (no protection)
        //
            ============================================================
```

```
51          vm.startPrank(someUser);
52          pool.swapExactOutput(
53              poolToken, // inputToken
54              weth, // outputToken
55              1e18, // outputAmount: user wants exactly 1 WETH
56              uint64(block.timestamp)
57          );
58
59          //
                ============================================================

60          //                  ASSERT THE DAMAGE
61          //
                ============================================================

62          uint256 userPoolTokenBalanceAfter = poolToken.balanceOf(
                someUser);
63          uint256 actualAmountPaid = userPoolTokenBalanceBefore -
64              userPoolTokenBalanceAfter;
65
66          console.log("Expected to pay (pre-market-move): ",
                expectedMaxInput);
67          console.log("Actually paid:                      ",
                actualAmountPaid);
68
69          // User received their 1 WETH...
70          assertEq(weth.balanceOf(someUser), 1e18);
71          assertGt(
72              actualAmountPaid,
73              expectedMaxInput);
74          vm.stopPrank();
75      }
```

Recommended Mitigation: We should include a maxInputAmount so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
1
2   function swapExactOutput(
3           IERC20 inputToken,
4   +       uint256 maxInputAmount,
5   .
6   .
7   .
8           inputAmount = getInputAmountBasedOnOutput(outputAmount,
                inputReserves, outputReserves);
9   +       if(inputAmount > maxInputAmount){
10  +           revert();
11  +       }
12          _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount. This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:
1. User calls `sellPoolTokens` with 10 pool tokens. 2. Expected output is computed using the `getOutputAmountBasedOnInput` function. 3. Actual output is computed using the `swapExactOutput` function. 4. Both outcomes are then compared to check if they match.

Paste the code below in the `TSwapPool.t.sol` file

Code

```
1
2  function testSellPoolTokensReturnsIncorrectAmountOfTokens()
3          public
4          depositLiquidity
5      {
6          address someUser = makeAddr("someUser");
7          uint256 userInitialPoolTokenBalance = 300e18;
8          poolToken.mint(someUser, userInitialPoolTokenBalance);
9          vm.startPrank(someUser);
10
11         poolToken.approve(address(pool), type(uint256).max);
12         uint256 wethBalanceBefore = weth.balanceOf(someUser);
13
14         pool.sellPoolTokens(10e18);
15
16         uint256 wethBalanceAfter = weth.balanceOf(someUser);
17         uint256 actualWethReceived = wethBalanceAfter -
               wethBalanceBefore;
18         uint256 expectedOutcome = pool.getOutputAmountBasedOnInput(
19             userInitialPoolTokenBalance,
20             poolToken.balanceOf(address(pool)),
21             weth.balanceOf(address(pool))
22         );
23
24         console.log("Actual WETH received:  ", actualWethReceived);
25         console.log("Expected WETH out:     ", expectedOutcome);
```

```
26
27          assert(actualWethReceived != expectedOutcome);
28          vm.stopPrank();
29      }
```

Recommended Mitigation: Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolTokens` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`)

```
1   function sellPoolTokens(
2           uint256 poolTokenAmount,
3   +       uint256 minWethToReceive,
4           ) external returns (uint256 wethAmount) {
5   -        return swapExactOutput(i_poolToken, i_wethToken,
        poolTokenAmount, uint64(block.timestamp));
6   +        return swapExactInput(i_poolToken, poolTokenAmount,
        i_wethToken, minWethToReceive, uint64(block.timestamp));
7       }
```

[H-4] In `TSwapPool::_swap` the extra tokens given to users after every swapCount breaks the protocol invariant of `x * y = k`

Description: The protocol follows a strict invariant of `x * y = k`. Where:

x: The balance of the pool token y: The balance of WETH k: The constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the k. However, this is broken due to the extra incentive in the _swap function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
1
2   swap_count++;
3   if (swap_count >= SWAP_COUNT_MAX) {
4     swap_count = 0;
5     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
6   }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol. Most simply put, the protocol's core invariant is broken

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens 2. That user continues to swap until all the protocol funds are drained

Paste the code below in the `TSwapPool.t.sol` file

Code

```
function testInvariantBroken() public {
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), 500e18);
        poolToken.approve(address(pool), 500e18);
        pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
        vm.stopPrank();

        uint256 outputWeth = 1e17;

        vm.startPrank(user);
        poolToken.mint(user, 100e18);
        poolToken.approve(address(pool), type(uint256).max);

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
```

```
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        int256 startingX = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaX = int256(outputWeth) * -1;

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        vm.stopPrank();
        int256 endingX = int256(weth.balanceOf(address(pool)));
        int256 actualDeltaX = int256(endingX) - int256(startingX);
        assertEq(actualDeltaX, expectedDeltaX);
    }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the x * y = k protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
-         swap_count++;
-         // Fee-on-transfer
-         if (swap_count >= SWAP_COUNT_MAX) {
-             swap_count = 0;
-             outputToken.safeTransfer(msg.sender, 1
```

```
        _000_000_000_000_000_000);
-         }
```

[H-5] Rebase, fee-on-transfer, and ERC777 tokens break protocol invariant

Description: Whenever there is a `swap` and tokens are sent out, there is a fee on transfer. However, this fee is not accounted for in the protocol invariant.Therefore breaking the protocol invariant (`x * y = k`). This is likely due to the fee-on-transfer mechanism present in the type of Token used.

```

 swap_count++;
        if (swap_count >= SWAP_COUNT_MAX) {
            swap_count = 0;
            outputToken.safeTransfer(msg.sender, 1
                _000_000_000_000_000_000);
        }
```

Impact: This breaks the protocol's invariant, and can lead to protocol funds being drained.

Proof of Concept: Due to the fee-on-transfer mechanism present in the type of Token used, any user that swaps more than 10 times gets an incentive, which is 1_000_000_000_000_000_000 tokens.

Paste the code below in the `TSwapPool.t.sol` file

Code

```

function testInvariantBroken() public {
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), 500e18);
        poolToken.approve(address(pool), 500e18);
        pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
        vm.stopPrank();

        uint256 outputWeth = 1e17;

        vm.startPrank(user);
        poolToken.mint(user, 100e18);
        poolToken.approve(address(pool), type(uint256).max);

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
```

```
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
```

```
            uint64(block.timestamp)
        );

        int256 startingX = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaX = int256(outputWeth) * -1;

        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        vm.stopPrank();
        int256 endingX = int256(weth.balanceOf(address(pool)));
        int256 actualDeltaX = int256(endingX) - int256(startingX);
        assertEq(actualDeltaX, expectedDeltaX);
    }
```

Recommended Mitigation: Change the kind of token used, or remove the fee-on-transfer mechanism.

```
-         swap_count++;
-         // Fee-on-transfer
-         if (swap_count >= SWAP_COUNT_MAX) {
-             swap_count = 0;
-             outputToken.safeTransfer(msg.sender, 1
    _000_000_000_000_000_000);
-         }
```

Medium

[M-1] `TSwapPool::deposit` is missing the deadline check causing the transaction to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is "The deadline for the transaction to be completed by". However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused. Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning. –> src/TSwapPool.sol:96:9: | 96 | uint64 deadline | ^^^^^^^^^^^^^^^^

Recommended Mitigation: Consider making the following change to the function:

```
function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
+       revertIfDeadlinePassed(deadline)
        revertIfZero(wethToDeposit)
        returns (uint256 liquidityTokensToMint)
    {...}
```

Low

[L-1] The LiquidityAdded event is indexed in the wrong order

Description: What the LiquidityAdded event is emitted in the `TSwapPool::_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The poolTokensToDeposit value should go in the third parameter position, whereas the wethToDeposit value should go second.

Impact: Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Proof of Concept:

Recommended Mitigation:

```
-       emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+       emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value output it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Proof of Concept: Below is my Proof of code, Paste this in the TSwapPool.t.sol file.

Code

```
function testSwapExactInputAlwaysReturnsZero() public depositLiquidity {
        address someUser = makeAddr("someUser");
        uint256 userInitialPoolTokenBalance = 11e18;
        poolToken.mint(someUser, userInitialPoolTokenBalance);
        vm.startPrank(someUser);

        poolToken.approve(address(pool), type(uint256).max);
        uint256 ouput = pool.swapExactInput(
            poolToken,
            1 ether,
            weth,
            0,
            uint64(block.timestamp)
        );

        // assertEq(poolToken.balanceOf(someUser), 0);
        assertEq(ouput, 0);
        vm.stopPrank();
    }
```

Recommended Mitigation:

```
-       uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
+       output = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
-       if (output < minOutputAmount) {
-           revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
+       if (output < minOutputAmount) {
+           revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
        }
-       _swap(inputToken, inputAmount, outputToken, outputAmount);
+       _swap(inputToken, inputAmount, outputToken, output);
}
```

Informationals

[I-1] `PoolFactory::PoolFactory___PoolDoesNotExist` is not used

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Lacking Zero address checks

```
constructor(address wethToken) {

+    if(wethToken == address(0)){
+    revert();
+}
         i_wethToken = wethToken;
     }
```

[I-3] PoolFactory::createPool should use .symbol() instead of .name()

```

- string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).name());
+ string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).symbol());
```

[I-4] TSwapPool::constructor Lacking zero address check - wethToken & poolToken

constructor(address poolToken, address wethToken, string memory liquidityTokenName, string memory liquidityTokenSymbol) ERC20(liquidityTokenName, liquidityTokenSymbol) { + if(wethToken || poolToken == address(0)){ + revert(); + } i_wethToken = IERC20(wethToken); i_poolToken = IERC20(poolToken); }

[I-5] TSwapPool events should be indexed

```
- event Swap(address indexed swapper, IERC20 tokenIn, uint256
    amountTokenIn, IERC20 tokenOut, uint256 amountTokenOut);
+ event Swap(address indexed swapper, IERC20 indexed tokenIn, uint256
    amountTokenIn, IERC20 indexed tokenOut, uint256 amountTokenOut);
```

[I-6] `poolTokenReserves` variable in the `TSwapPool::deposit` function is unused therefore should be removed

```
 function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
```

```
7           external
8           revertIfZero(wethToDeposit)
9           returns (uint256 liquidityTokensToMint)
10      {
11          if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
12              revert TSwapPool__WethDepositAmountTooLow(
13                  MINIMUM_WETH_LIQUIDITY,
14                  wethToDeposit
15              );
16          }
17          if (totalLiquidityTokenSupply() > 0) {
18              uint256 wethReserves = i_wethToken.balanceOf(address(this))
                    ;
19  -            uint256 poolTokenReserves = i_poolToken.balanceOf(address(
        this));
20          }
21      }
```

[I-7] MINIMUM_WETH_LIQUIDITY is a constant, therefore there is no need for it to be emitted in the revert error.

```
1  if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
2              revert TSwapPool__WethDepositAmountTooLow(
3  -               MINIMUM_WETH_LIQUIDITY,
4                  wethToDeposit
5              );
6  }
```

[I-8] `liquidityTokensToMint` should be above the _addLiquidityMintAndTransfer function to follow CEI

```
1  +            uint256 liquidityTokensToMint;
2  _addLiquidityMintAndTransfer(
3                  wethToDeposit,
4                  maximumPoolTokensToDeposit,
5                  wethToDeposit
6              );
7  -            liquidityTokensToMint = wethToDeposit;
```